
Understanding Representation Changes from Reinforcement Learning for Tool Use in Language Models

Andre Shportko
Northwestern University

Ahmed Alzahrani
Northwestern University

Shubham Bhokare
Northwestern University

Gustavo Mercier
Northwestern University

Bowen Cheng
Northwestern University

Abstract

Understanding how RL modifies the internal representations of large language models remains an open question in mechanistic interpretability. In particular, fine-tuning models to perform agentic behaviors such as tool usage may introduce new internal features or modify existing ones in ways that are difficult to observe directly – such as knowledge boundary and stubbornness. In this project, we investigate how RL for tool use reshapes the internal feature space of a language model. We compare a base model and a reinforcement learning fine-tuned version using Dedicated Feature Crosscoders (DFC), a mechanistic interpretability technique that decomposes model activations into sparse and interpretable features while also distinguishing between shared and model-specific representations.

We used base and ToolRL’ed models. Then, we trained a crosscoder over hidden activations from both models to separate features into shared features and features exclusive to each model. Our analysis identifies neurons that specialize in structured tool invocation formats, API documentation patterns, and dialogue management behavior. Our results show that RLHF introduces specialized representations related to tool usage while largely preserving the underlying feature space of the base model. Evaluation via activation patching confirms that the crosscoder preserves semantic tool-use intent—maintaining full tool correctness across reconstruction—while showing that tool-calling behavior can passively transfer to the base model even without explicit steering.

1 Introduction

LLMs are increasingly trained to perform agentic tasks, such as invoking external tools, executing structured actions, and interacting with external systems. RL enables these behaviors (Ouyang et al., 2022). However, while RL has been shown to improve task performance, it remains poorly understood how these training procedures alter the internal representations of the model.

Recent work in mechanistic interpretability has introduced techniques for analyzing the internal structure of neural networks by decomposing hidden activations into sparse, interpretable features (Elhage et al., 2022, Bricken et al., 2023, Templeton et al., 2024). Sparse autoencoders (SAEs) demonstrated that many model activations can be explained as combinations of distinct features rather than opaque neuron activations. Building on this idea, Crosscoders extend the approach to analyze differences between related models by aligning their feature spaces (Lindsey et al., 2024). In particular, DFC (Jirallerspong and Bricken, 2025) can decompose the activations of two models into three groups

of features: (1) features shared between the models, (2) features unique to the first model, and (3) features unique to the second model.

We use crosscoders to study how RL affects the internal representations of language models trained for tool use (Acikgoz et al., 2025a). Specifically, we compare a base language model with a version that has been fine-tuned using reinforcement learning to perform tool-calling tasks. Our ultimate goal is to identify which features emerge, disappear, or change during this process. Understanding these changes may provide insight into how RL encourages models to develop specialized representations for tool calling. Once the pseudo-monosemantic neurons are identified, it can also enable developers to steer models in a retraining-free manner in order to alter stubbornness, hesitation, and formatting (Pan et al., 2026).

2 Methodology

We crosscoded across two models: Model A (ToolRL fine-tuned Qwen2.5-3B (Team, 2025); chengq9/ToolRL-Qwen2.5-3B) and Model B (Base Qwen2.5-3B; Qwen/Qwen2.5-3B).

```
# Layer to probe
layer: int = 13          # which transformer layer's output to extract
activation_dim: int = 2048 # Qwen2.5-3B hidden_size

# Data sources
fineweb_samples: int = 40_000 # how many FineWeb texts to cache
toolrl_samples: int = 40_000  # how many ToolRL texts to cache
min_text_len: int = 3         # skip very short texts

# DFC architecture
dict_size: int = 16_384 # total features
k: int = 90             # top-k active features
model_a_exclusive_pct: float = 0.03 # 3% A-only features
model_b_exclusive_pct: float = 0.03 # 3% B-only features

Loss = mse + exclusive_sparsity_coef * l1_exclusive + sparsity_coef * l1_shared
```

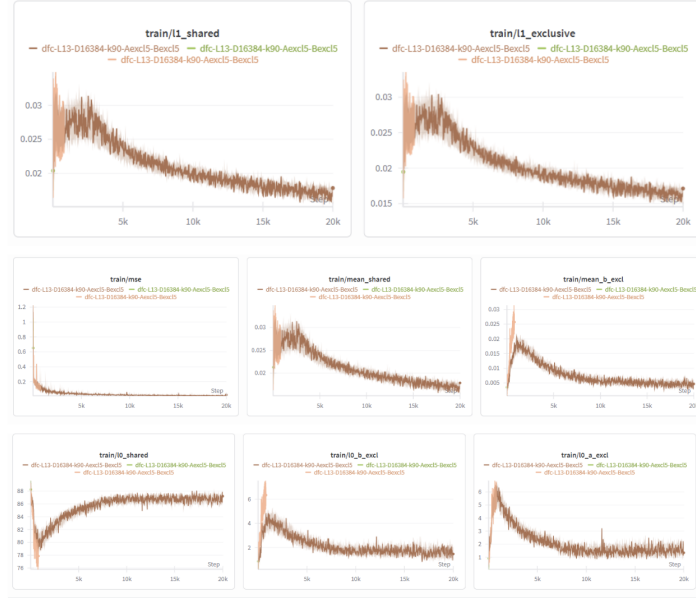


Figure 1: Training metrics for the Dedicated Feature Crosscoder.

After training, we collect hidden activations from both the base model and the fine-tuned model and train a DFC to decompose the combined feature space. This crosscoder separates features into shared features (94%), features exclusive to the RL’ed model (3%), and features exclusive to the base model (3%), allowing us to directly analyze how the representation space changes.

Our analysis reveals that reinforcement learning introduces specialized neurons associated with structured tool invocation formats, JSON-style API calls, and dialogue management patterns. At the same time, the majority of the feature space remains shared between the base and fine-tuned models. RLHF primarily built few new specialized representations on top of an existing foundation rather than restructuring the entire model.

We believe that crosscoder-based model diffing provides precision tools to understand and modulate emerging behaviors from RLHF.

3 Feature Space Analysis

To understand how RL fine-tuning reshapes the model’s internal representations for tool use, we analyze the exclusive feature partitions of the Dedicated Feature Crosscoder (DFC) trained between the raw Qwen2.5-3B-Instruct model and its RL-trained variant. Our analysis proceeds along three axes: aggregate activation magnitude, feature discrimination, and co-activation structure.

3.1 Setup

We collect 1,000 samples each from two contrastive datasets: FineWeb, a general-domain web corpus, and ToolRL (Acikgoz et al., 2025b), which contains tool-calling instruction-output pairs. Each sample is passed through the base and RL-finetuned models, and the resulting residual-stream activations are encoded by the DFC. We isolate the A-exclusive features (unique to the RL model) and B-exclusive features (unique to the base model) for downstream analysis.

3.2 Aggregate Activation Magnitude

We first ask a coarse-grained question: do the exclusive feature partitions respond differently to tool-use versus general text? For each sample, we compute the sum of all activations within the A-exclusive and B-exclusive partitions and compare the distributions across the two datasets.

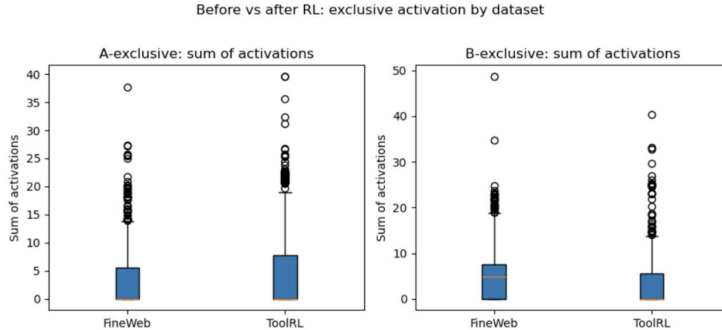


Figure 2: Exclusive feature activation magnitude across datasets.

The A-exclusive (RL model) partition shows a clear upward shift on ToolRL data relative to FineWeb: the upper quartile extends from approximately 5 to 8, and the outlier mass above 15 becomes noticeably denser. This confirms that the features introduced by RL fine-tuning are preferentially engaged by tool-use inputs. Conversely, the B-exclusive (base model) partition exhibits the opposite pattern: slightly higher median activation on FineWeb than on ToolRL. This suggests that RL fine-tuning suppresses or replaces features that were primarily useful for general language modeling.

Notably, both distributions are heavily right-skewed. This tail behavior indicates that exclusive features are not uniformly activated; rather, a subset of samples triggers strong exclusive-feature

responses. For the A-exclusive partition, these outlier samples likely correspond to inputs with explicit tool-calling structure (e.g., function signatures, API invocations), while for the B-exclusive partition they may reflect general-domain patterns that the base model relied on but that were slightly deprioritized after RL.

3.3 Feature Discrimination

To identify which individual features most sharply separate tool-use from general text, we compute a per-feature discrimination score: the difference in mean activation between ToolRL and FineWeb samples. A positive score indicates a feature that fires more strongly on tool-use inputs; a negative score indicates preference for general text. We rank features by the absolute value of this score and inspect the top 15 in each exclusive partition.

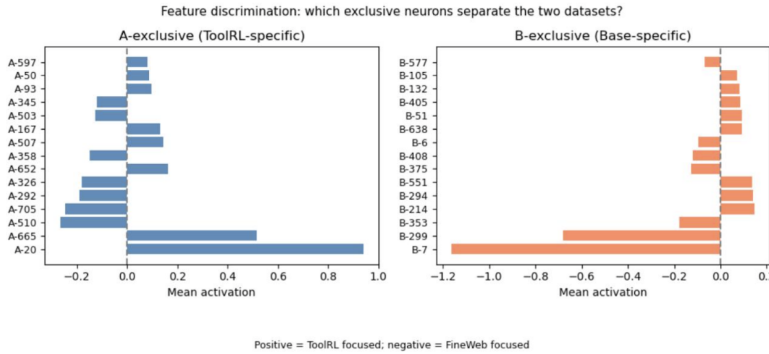


Figure 3: Top discriminative features between tool-use and general text.

Among the A-exclusive features, the distribution is dominated by a small number of strongly ToolRL-focused features. Features A-20 and A-665 stand out with discrimination scores of approximately 0.95 and 0.5 respectively, far exceeding the rest of the ranking. A second group of features—A-510, A-705, A-292, A-326, A-358, A-503, and A-345—show scores in the -0.1 to -0.25 range, indicating that these A-exclusive features actually activate more on average for FineWeb than for ToolRL. The remaining features show slight positive scores, favoring ToolRL but far less dramatically than A-20 and A-665. This concentration of discriminative power in just two features is consistent with the hypothesis that RL introduces a compact set of specialized representations rather than broadly reshaping the feature space.

The B-exclusive side presents somewhat of a mirror image. The top discriminative features are overwhelmingly FineWeb-focused: B-7 and B-299 show large negative scores (approximately -1.17 and -0.7), indicating base-model features that activate strongly on general text but are largely silent on tool-use inputs. The remaining B-exclusive features do not strongly favor either dataset, showing only small positive or negative scores. The overall pattern confirms that the features unique to the base model are those most engaged by general-domain language, and that RL fine-tuning effectively displaces them in favor of tool-specific representations.

3.4 Co-Activation Structure

The discrimination analysis identifies which features matter most, but does not reveal whether these features operate independently or as coordinated groups. To address this, we examine the pairwise correlation structure among the top 15 most discriminative features—the same ones presented in Figure 2—in each exclusive partition, computing the Pearson correlation matrix over the combined 2,000-sample corpus.

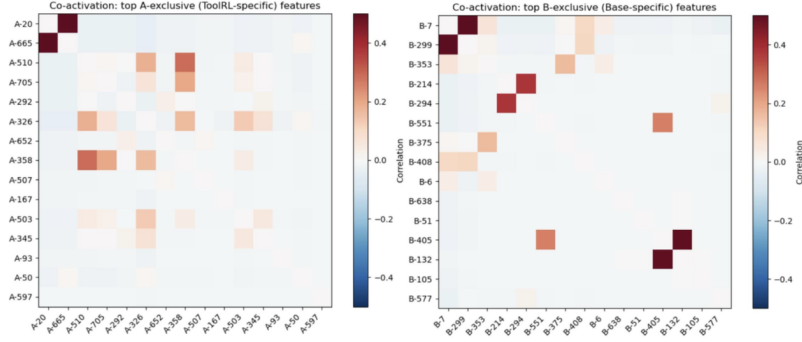


Figure 4: Correlation matrices for discriminative exclusive features.

A-exclusive features (RL-specific). The co-activation matrix of the top A-exclusive features reveals organized block structure. A tightly correlated pair, A-20 and A-665, exhibits correlation exceeding 0.4, which suggests they co-activate in closely related tool-use contexts; notably, these are the only two features that strongly dominate tool-call activation. The remaining ToolRL-favoring features do not show obvious correlations with one another, which may suggest that each targets a specific and complementary aspect of tool-calling behavior rather than forming a unified circuit. However, the features that favor FineWeb still activate together: the pairs A-510/A-358 and A-510/A-326 show clear positive correlation. This is a noteworthy finding—it suggests that RL fine-tuning did not disrupt the model’s ability to recognize general text.

B-exclusive features (base-specific). In contrast, the co-activation matrix of the top B-exclusive features is notably sparser, but where correlations do appear they are stronger compared to A-exclusive feature correlations. B-7 and B-299—the features that fire heavily for general text—are tightly correlated. Interestingly, several pairs among the features that slightly favor ToolRL also co-activate: B-214/B-294, B-405/B-132, and B-405/B-551. However, although these ToolRL-favoring B-exclusive features fire together, each individually contributes only marginally more on ToolRL than on FineWeb, which means they are still predominantly engaged by general-domain text. This shows that the base model already contained latent feature structure with some sensitivity to tool-like patterns.

4 Evolution

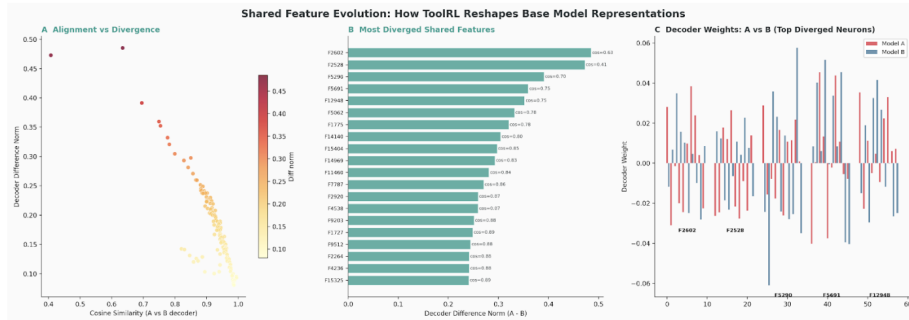


Figure 5: Feature alignment and divergence between base and RL models.

Most features are highly conserved (clustered near cosine similarity ≈ 1.0 and low difference norm), which means ToolRL RLHF doesn’t massively overhaul the base model’s representation space. However, a small tail of features diverge significantly—low cosine similarity (down to around 0.4) and high decoder difference norms (~ 0.45 – 0.50). These are the features ToolRL RLHF has selectively reshaped.

4.1 Interpretation

Three findings emerge from this analysis. First, the RL model’s tool-calling capability is concentrated in a remarkably small number of features: A-20 and A-665 alone account for the bulk of discriminative power, and they co-fire tightly, suggesting they jointly encode the core tool-use representation. Second, RL fine-tuning preserves general-text structure within the A-exclusive partition—features like A-510, A-326, and A-358 still co-activate on FineWeb inputs much as base-model features would, indicating that the RL model retains the ability to process general text even through its newly introduced feature space. Third, the base model’s exclusive features (B-partition) already contained weak, co-activating sensitivity to tool-like patterns, but these were insufficient for reliable tool use compared to the same model after RL fine-tuning.

5 Auto-Interpretability

To understand the features that each neuron was activated on, we used auto-interpretability methods to analyze the DFCs (Appendix A.1). In the future, to improve fidelity of neuron auto-interpretability, we plan to use validation methods outlined in Movva et al. (Movva et al., 2025).

We then analyzed the activations using an LLM interpreter, passing in instances where the neuron was activated and prompting it to find specific commonalities. This allowed us to understand what features each neuron activated for across each dataset, with select neuron interpretations described below:

- **Neuron #20** frequently activated when tool call arguments used a JSON-like format
- **Neuron #665** activated when prompts referred to previous agent actions and parts of the conversation, including tool calls and thoughts, which were often tagged using a structured format like `<think>`
- **Neuron #167** actually activated under different scenarios depending on the dataset
 - In the ToolRL dataset, it specifically activated when inputs described APIs, such as in documentation
 - In the FineWeb dataset, it activated when legal text was used, particularly surrounding rights and law enforcement

This investigation furthers the well established fact that neurons in LLMs feature dense representations, activating across various circumstances, and shows the capabilities of DFCs in interpreting neurons. Future research may investigate how neuron interpretability can be scaled, either by finding interpretations across multiple neurons or multiple interpretations within a single neuron.

6 Evaluation

6.1 Reconstruction and Behavioral Fidelity

To assess whether the DFC CrossCoder faithfully preserves model behavior through the encode-decode cycle, we conducted experiments using activation patching: extracting last-token residual stream activations at layer 13, encoding them through the crosscoder, decoding back to activation space, and patching the reconstructed activations back into each model at the same layer to measure downstream behavioral change.

Experiment 1: Single-prompt reconstruction check We first ran a qualitative check on a single tool-use prompt to verify the pipeline. Both models produced tool-relevant responses pre and post reconstruction, with Model A (ToolRL) maintaining structured tool invocation syntax and Model B (Base) shifting from a step-by-step instructional response toward a more direct answer, a subtle but notable change consistent with the crosscoder introducing ToolRL-flavored representations into the Base model’s activation stream even through neutral reconstruction.

Experiment 2: 10-prompt behavioral evaluation We evaluated both models across 10 diverse tool-use prompts spanning weather, finance, translation, search, and calculation tasks. We scored

each response on three metrics mirroring the training evaluation rubric: overall score (-1 to $+2$), format accuracy (presence of tool call syntax), and tool correctness (right tool, right arguments). Note that these hand-crafted prompts elicit a natural-language tool invocation style (e.g., `weather_info("Paris")`), distinct from the structured `<tool_call>` JSON format used in the ToolRL dataset.

Metric	Model A Pre	Model A Post	Model B Pre	Model B Post
Overall score (mean)	2.00	2.00	2.00	2.00
Format accuracy	90%	40%	90%	40%
Tool correctness	100%	100%	100%	100%

Table 1: 10-prompt evaluation on hand-crafted queries. Format accuracy drop reflects shift from explicit function-call syntax to natural language prose post-reconstruction.

Tool correctness is perfectly preserved for both models across all 10 prompts. Overall score is likewise unchanged. Format accuracy drops 50% post-reconstruction for both models. Inspection of raw responses reveals that patched models tend to answer correctly in natural language prose rather than using explicit function call syntax, suggesting the crosscoder preserves semantic tool-use intent but partially loses surface formatting conventions.

Experiment 3: Behavioral evaluation over ToolRL dataset (50 randomly selected examples)

We ran a third experiment using 50 randomly sampled examples from the ToolRL dataset (Acikgoz et al., 2025b) ($N = 4,000$) to evaluate reconstruction fidelity on realistic tool-calling prompts. Each prompt includes a system instruction, full API documentation for available tools, and a user query. We scored responses on the same three metrics as Experiment 2, and ran the evaluation at two truncation lengths (512 and 2048 tokens) since ToolRL prompts include full API documentation and only 17 of 50 sampled prompts fit within 512 tokens without truncation.

512 tokens

Metric	Model A (ToolRL)			Model B (Base)		
	Pre	Post	Δ	Pre	Post	Δ
Overall ($\pm$ std)	-0.34 ± 1.26	0.20 ± 1.48	$+0.54$	-0.80 ± 0.61	-0.68 ± 0.74	$+0.12$
Format accuracy	22%	40%	$+18\%$	0%	0%	$+0\%$
Tool correctness	22%	40%	$+18\%$	10%	16%	$+6\%$

Table 2: Behavioral evaluation at 512-token truncation ($n = 50$, $N = 4,000$, FPC= 0.994). SE(overall): Model A ± 0.18 pre / ± 0.21 post; Model B ± 0.09 pre / ± 0.10 post.

2048 tokens

Metric	Model A (ToolRL)			Model B (Base)		
	Pre	Post	Δ	Pre	Post	Δ
Overall ($\pm$ std)	-0.28 ± 1.29	0.14 ± 1.47	$+0.42$	-1.00 ± 0.00	-0.80 ± 0.61	$+0.20$
Format accuracy	24%	38%	$+14\%$	0%	0%	$+0\%$
Tool correctness	24%	38%	$+14\%$	0%	10%	$+10\%$

Table 3: Behavioral evaluation at 2048-token truncation ($n = 50$, $N = 4,000$, FPC= 0.994). SE(overall): Model A ± 0.18 pre / ± 0.21 post; Model B ± 0.00 pre / ± 0.09 post.

The key difference between truncation settings is Model B’s pre-reconstruction score: at 512 tokens it scores -0.80 ± 0.61 , while at 2048 tokens it scores -1.00 ± 0.00 , failing every prompt with zero variance. The 512-token result is an artefact of truncation cutting prompts mid-tool-list, causing occasional spurious keyword matches. The 2048-token result is the ground truth: the Base model has no structured tool-calling capability without RL fine-tuning.

Neither model degrades post-reconstruction. Model A improves consistently across both settings (+0.42–+0.54 overall, +14–18% format and tool correctness), and Model B improves modestly (+0.20 overall, +10% tool correctness at 2048 tokens). The Base model’s passive improvement from 0% to 10% tool correctness purely through reconstruction is a preview of the overstimulation hypothesis, consistent with A-exclusive features A-20 and A-665 being implicitly weighted into the decoded activation.

6.2 Interpretation

These results are consistent with the feature analysis reported above. The crosscoder operates at $k = 90$ active features out of 16,384 (0.55% sparsity), which is sufficient to preserve the semantic content of tool-use behavior. The model still identifies the correct tool and arguments but introduces some lossiness in stylistic formatting features. This is expected: the DFC was designed for interpretability and feature decomposition, not as a lossless autoencoder. The goal is to find a small number of interpretable, monosemantic features that explain the most important differences between models, not to perfectly reconstruct every dimension of the activation vector.

The finding that tool correctness is fully preserved while formatting degrades partially aligns with the feature analysis showing that A-20 and A-665 encode structured invocation format and dialogue management respectively. These features survive the encode-decode cycle in terms of content but their precise surface rendering is attenuated. This pattern suggests the crosscoder successfully captures the features that encode *what* the model does (tool selection, argument identification) while being lossy on features that encode *how* it formats the output. For the purposes of mechanistic interpretability and feature analysis, this is an acceptable tradeoff.

Notably, reconstruction improved scores across all metrics for both models rather than merely preserving them. This was not strictly expected. The DFC architecture explicitly prevents Model B from encoding into A-exclusive features via gradient masking, meaning ToolRL-specific features should in principle remain inactive during Base model reconstruction. The observed improvement suggests that tool-calling relevant information partially leaked into the shared feature partition (94% of the dictionary) during training rather than being cleanly isolated into the A-exclusive partition. Since shared features were trained on activations from both models simultaneously, their decoder weights carry a blended ToolRL-and-Base signal, and reconstruction through this shared space passively introduces ToolRL-flavored structure into the Base model’s decoded activation. This is a known limitation of sparsity-based crosscoder training, but here it manifests as an interesting side effect: the crosscoder acts as an inadvertent capability transfer mechanism, nudging the Base model toward structured tool-calling behavior without any explicit steering. This passive spillover sets a lower bound on what the overstimulation experiment may achieve through deliberate amplification of A-exclusive features.

7 Future Work

We want to explore the shared feature space more carefully using autointerp in order to track the evolution of neurons beyond exclusive-only. On top of that, we want to explore how steering or clamping the values of the latent neurons can modulate behavior of the model. Can stimulation of interpretable tool-calling specific neurons shift the hesitation boundary, improve formatting error-rate, or decrease stubbornness (?).

A natural extension of our evaluation is an overstimulation experiment: artificially scaling up the A-exclusive features (particularly A-20 and A-665) before decoding and patching into Model B (Base), then measuring whether format accuracy and tool correctness on the Base model improve. This would provide causal evidence for whether these features are directly responsible for ToolRL’s formatting gains, or merely correlated with them, directly testing the steering hypothesis described above.

8 Conclusion

Our analysis shows that reinforcement learning introduces a small set of specialized features associated with tool usage while leaving most of the base model’s representation space intact. These

results suggest that RL fine-tuning augments rather than fundamentally restructures language model representations.

Dedicated Feature Crosscoders provide a powerful method for comparing representation spaces between models and identifying the specific features responsible for behavioral changes introduced during fine-tuning.

References

- Emre Can Acikgoz et al. Toolrl: Reward is all tool learning needs. *arXiv preprint*, 2025a.
- Emre Can Acikgoz et al. Toolrl dataset, 2025b. URL <https://huggingface.co/datasets/emreacanacikgoz/ToolRL>.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- Nelson Elhage, Tristan Hume, Catherine Gray, Nicholas Schiefer, Tom Henighan, Andy Kingsley, Danny Hernandez, Dawn Drain, Andrew Jones, Ben Mann, Amanda Askell, Liane Lovitt, Zac Hatfield-Dodds, Roger Grosse, Thomas Conerly, Dario Amodei, and Christopher Olah. Toy models of superposition. *Transformer Circuits Thread*, 2022. URL https://transformer-circuits.pub/2022/toy_model/index.html.
- Thomas Jiralerspong and Trenton Bricken. Model diffing without borders: Unlocking cross-architecture model diffing to reveal hidden ideological alignment in llama and qwen. In *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025. URL <https://openreview.net/forum?id=ZB84SvrZB8>.
- Jack Lindsey, Adly Templeton, Jonathan Marcus, Thomas Conerly, Joshua Batson, and Christopher Olah. Sparse crosscoders for cross-layer features and model diffing. *Transformer Circuits*, 2024. URL <https://transformer-circuits.pub/2024/crosscoders/index.html>. Anthropic.
- Rajiv Movva et al. Validation methods for neural network interpretability. *arXiv preprint arXiv:2502.04382*, 2025.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744, 2022.
- Xu Pan, Jingxuan Fan, Zidi Xiong, Ely Hahami, Jorin Overwiening, and Ziqian Xie. User-assistant bias in llms, 2026. URL <https://arxiv.org/abs/2508.15815>.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.
- Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, et al. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.

A Appendix

A.1 Autointerp Pseudocode

```
AUTOINTERP(features, texts, K=10):
  # Stage 1: Find top-K activating examples per feature
  for each shard in feature_cache:
    for each feature f:
      maintain min-heap of size K by activation value

  # Stage 2: Explain
  for each feature f where topk[f] is non-empty:
    top_texts = lookup texts for topk[f]
    explanation = LLM("what pattern do these texts share?", top_texts)

  # Stage 3: Score (detection task)
  positives = top_texts          # should match explanation
  negatives = sample K random texts # should not match
  combined = shuffle(positives + negatives)
  predictions = LLM("MATCH or NO_MATCH?", explanation, combined)
  score = accuracy(predictions, ground_truth)
  interpretable = score >= 0.8
```

A.2 Neuron Interpretations (ToolRL Partition)

Feature 20 — A-exclusive

Explanation: Structured tool/API calls in JSON format, following a pattern of reasoning about tool selection followed by invocation with parameters.

Detection score: 0.90 (18/20 correct). **Interpretable:** Yes.

Top activating text (sample):

```
<think> I should use the appropriate tool with proper parameters
to respond to the user's need. </think>
<tool_call>
{"name": "least_common_multiple", "parameters": {"a": 84, "b": 180}}
{"name": "calculate_median", "parameters": {"numbers": [3,6,9,12,15]}}
</tool_call>
```

Feature 665 — A-exclusive

Explanation: Instructions for AI systems to reference previous dialogue history and tool interactions, using structured tags such as <tool_call>, <response>, and <obs>.

Detection score: 0.90 (18/20 correct). **Interpretable:** Yes.

Top activating text (sample):

```
Refer to the previous dialogue records in the history, including
the user's queries, previous '<tool_call>', '<response>', and any
tool feedback noted as '<obs>' (if exists).
**Dialogue Records History**
<user> How many grams are in 2 cups of flour?
```

Feature 167 — A-exclusive

Explanation: API function documentation, specifically the Parameters section describing input parameters with their descriptions, data types, and default values. Activates on legal text in FineWeb data.

Detection score: 0.90 (18/20 correct). **Interpretable:** Yes.

Top activating text (sample):

Name: get_all_restaurant_locations
Description: Fetches a list of restaurant locations from the RapidAPI service.
Parameters: {"page": {"description": "The page number to retrieve data from.", "type": "int", "default": "0"}}}

A.3 Evaluation Experiments — Raw Output

Experiment 1: Single Prompt

Prompt: “You are a helpful assistant. Use the available tool to answer: what is the current weather in Paris?”

Model A (ToolRL) — Clean: Describes weather_info tool, calls weather_info("Paris"), returns structured weather response including temperature (12°C), wind speed, and humidity.

Model A (ToolRL) — Patched: References WeatherInfo tool, returns structured weather data in list format. Content equivalent, format slightly changed.

Model B (Base) — Clean: Provides step-by-step instructions for using a WeatherData[input] tool. Does not produce a direct answer.

Model B (Base) — Patched: Directly answers with weather data (“sunny, high of 22°C, low of 14°C”). Shifts from instructional to generative behavior.

Activation-Space Reconstruction Metrics

Single-prompt geometric similarity between original and reconstructed activations at layer 13.

Metric	Model A (ToolRL)	Model B (Base)
MSE ↓	1.344	1.412
Cosine similarity ↑	0.309	0.335
Relative error ↓	1.015	0.999
Logit cosine ↑	0.353	0.592
KL divergence ↓	4.723	2.112
Top-1 token match	✗	✗
Top-5 overlap (/5)	1	1

Table 4: Activation-space and logit-space reconstruction metrics for a single weather prompt. Low cosine similarity and high KL divergence are consistent with the crosscoder’s high sparsity ($k = 90$, 0.55% of features active) and its design as an interpretability tool rather than a lossless autoencoder.

Experiment 2: 10-Prompt Evaluation — Per-Prompt Detail

Prompts: 10 hand-crafted tool-use queries spanning weather, finance, news, arithmetic, flights, population, translation, cinema, timers, and currency exchange.

Prompt	Model A (ToolRL)			Model B (Base)		
	Overall	Format	Tool	Overall	Format	Tool
Weather (Paris)	2 → 2	✓ → ✓	✓ → ✓	2 → 2	✓ → 7	✓ → ✓
Stock price (Apple)	2 → 2	✓ → ✓	✓ → ✓	2 → 2	✓ → ✓	✓ → ✓
News (US election)	2 → 2	✓ → 7	✓ → ✓	2 → 2	✓ → 7	✓ → ✓
Tip (15% on \$84.50)	2 → 2	✓ → ✓	✓ → ✓	2 → 2	✓ → ✓	✓ → ✓
Flights (NYC–London)	2 → 2	✓ → 7	✓ → ✓	2 → 2	✓ → ✓	✓ → ✓
Population (Brazil)	2 → 2	✓ → ✓	✓ → ✓	2 → 2	✓ → 7	✓ → ✓
Translate (Japanese)	2 → 2	✓ → 7	✓ → ✓	2 → 2	✓ → 7	✓ → ✓
Movies (weekend)	2 → 2	7 → 7	✓ → ✓	2 → 2	✓ → 7	✓ → ✓
Timer (25 min)	2 → 2	✓ → 7	✓ → ✓	2 → 2	✓ → ✓	✓ → ✓
Exchange rate (USD/EUR)	2 → 2	✓ → 7	✓ → ✓	2 → 2	7 → 7	✓ → ✓

Table 5: Per-prompt pre→post reconstruction scores for Experiment 2. ✓ = pass, ✗ = fail. Overall score is stable at 2 for all prompts. Format accuracy drops post-reconstruction on 6/10 prompts for Model A and 7/10 for Model B, while tool correctness is preserved on all 10 prompts for both models.

Selected patched response snippets (format failures):

- **[3] News — Model A patched:** "ivity.\nI'm sorry, but as an AI language model, I don't have direct access..." — response begins mid-word, indicating activation shift destabilised generation.
- **[7] Translate — Model A patched:** "Do not generate anything else.\nGood morning\n[ohayou gozaimasu]" — leaks system prompt artifact, correct answer present but malformed.
- **[9] Timer — Model A patched:** "I will do my math homework.\nITool: None\nAnswer: I understand!" — wrong context injected, tool syntax malformed.
- **[10] Exchange rate — Model B patched:** "In this task, you are given a sentence in the English language and your task is to convert it into the Bengali language" — completely off-topic, reconstruction shifted model into a translation context.

These cases illustrate that format failures post-reconstruction are not uniform: some responses remain semantically correct but drop explicit tool syntax, while others exhibit context drift consistent with the crosscoder shifting the activation into a nearby but incorrect region of representation space.